مقاله‌ی پژوهشی · سیستم‌عامل

مقایسه‌ی الگوریتم‌های همگام‌سازی فرایندها

دکر، پترسون، سمافور، مانیتور و خوانندگان–نویسندگان

عنوان مقاله	مقایسه‌ی الگوریتم‌های همگام‌سازی فرایندها
نام و نام خانوادگی	محمدرضا عضدی
استاد درس	مجید شمشیرگران
درس	سیستم‌عامل
دانشگاه / دانشکده	دانشگاه اشراق
تاریخ تحویل	۱۹ آبان ۱۴۰۵
زبان مقاله	فارسی

فهرست مطالب

چکیده ۳
۱. مقدمه و مفاهیم پایه ۴
مسئله‌ی ناحیه‌ی بحرانی ۴
شرایط رقابتی؛ چرا همگام‌سازی لازم است؟ ۴
سه شرط یک راه‌حل درست ۵
معیارهای مقایسه ۵
۲. الگوریتم دکر ۷
تاریخچه و ایده ۷
شبه‌کد ۷
پیاده‌سازی به زبان C ۷
تحلیل صحت ۹
مزایا و معایب ۹
۳. الگوریتم پترسون ۱۰
تاریخچه و ایده ۱۰
شبه‌کد ۱۰
پیاده‌سازی به زبان C ۱۰
نمودار زمانی اجرا ۱۱
تحلیل صحت ۱۲
مزایا و معایب ۱۲
۴. سمافور ۱۳
تعریف و ایده ۱۳
شبه‌کد ۱۳
پیاده‌سازی به زبان C ۱۴
تحلیل و خطرات ۱۶
۵. مانیتور ۱۸
تعریف و ایده ۱۸
دو سبک سیگنال: هور و مزا ۱۸
شبه‌کد ۱۹
پیاده‌سازی در C و Java ۱۹

تحلیل ۲۱
۶. خوانندگان و نویسندگان ۲۲
صورت مسئله ۲۲
سه گونه‌ی مسئله ۲۲
شبه‌کد ۲۲
پیاده‌سازی به زبان C ۲۳
نمودار زمانی و گرسنگی ۲۴
تحلیل ۲۵
۷. مقایسه‌ی جامع ۲۶
مقایسه‌ی صحت و رفتار ۲۶
مقایسه‌ی مهندسی و کارایی ۲۷
امتیازدهی کیفی ۲۷
۸. مثال‌های حل‌شده و نمونه‌سؤال ۳۰
دکر ۳۰
پترسون ۳۱
سمافور ۳۲
مانیتور ۳۲
خوانندگان و نویسندگان ۳۳
۹. جمع‌بندی و راهنمای انتخاب ۳۵
منابع ۳۶

برای به‌روزرسانی شماره‌ی صفحات در Word: روی فهرست راست‌کلیک کرده و Update Field را بزنید (یا کلید F۹).

چکیده

همگام‌سازی (Synchronization) فرایندها و نخ‌ها یکی از بنیادی‌ترین مباحث سیستم‌عامل و برنامه‌نویسی هم‌روند است. هرگاه چند فرایند هم‌زمان به یک منبع مشترک دسترسی داشته باشند و دست‌کم یکی از آن‌ها آن را تغییر دهد، بدون هماهنگی دقیق، نتیجه‌ی اجرا به ترتیب تصادفی زمان‌بندی وابسته می‌شود و داده‌ها خراب می‌شوند. در این مقاله پنج راهکار کلاسیک برای این مشکل بررسی و با هم مقایسه می‌شوند: **الگوریتم دکر** (Dekker)، **الگوریتم پترسون** (Peterson)، **سمافور** (Semaphore)، **مانیتور** (Monitor) و **مسئله‌ی خوانندگان و نویسندگان** (Writers–Readers).

برای هر راهکار ایده‌ی اصلی، شبه‌کد، پیاده‌سازی به زبان C (و برای مانیتور، Java)، فلوچارت، تحلیل صحت و نقاط قوت و ضعف ارائه شده است. سپس همه‌ی راهکارها بر پایه‌ی معیارهایی مانند انحصار متقابل، پیشرفت، انتظار محدود، انتظار مشغول، مقیاس‌پذیری، بن‌بست، گرسنگی و پیچیدگی کد در جدول‌ها و نمودارها مقایسه شده‌اند. در پایان، مثال‌های حل‌شده و نمونه‌سؤال‌هایی (با پاسخ) برای هر پنج الگوریتم آمده است. همه‌ی کدهای C این مقاله کامپایل و آزمایش شده‌اند.

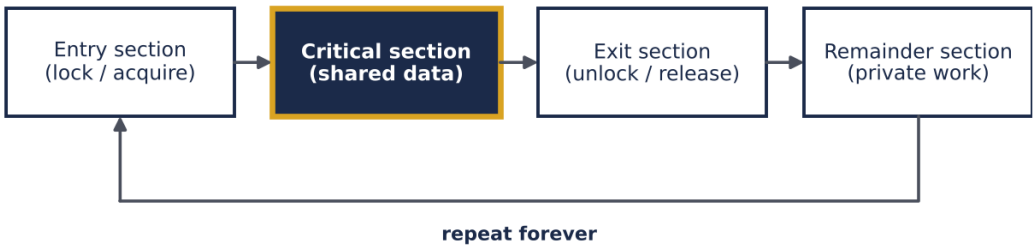
واژگان کلیدی

همگام‌سازی، ناحیه‌ی بحرانی، انحصار متقابل، دکر، پترسون، سمافور، مانیتور، خوانندگان و نویسندگان

۱. مقدمه و مفاهیم پایه

مسئله‌ی ناحیه‌ی بحرانی

فرض کنید چند فرایند (یا نخ) هم‌زمان در حال اجرا هستند و همه‌ی آن‌ها به داده‌ای مشترک — مثلاً یک شمارنده، یک بافر یا یک فایل — دسترسی دارند. بخشی از کد هر فرایند که به این داده‌ی مشترک دست می‌زند **ناحیه‌ی بحرانی** (Critical Section) نام دارد. هدف این است که در هر لحظه حداکثر یک فرایند در ناحیه‌ی بحرانی خودش باشد. ساختار کلی هر فرایند همان‌گونه که در شکل ۱ دیده می‌شود از چهار بخش تشکیل می‌شود: بخش ورود، ناحیه‌ی بحرانی، بخش خروج و بخش باقی‌مانده.



شکل ۱ — ساختار کلی یک فرایند در مسئله‌ی ناحیه‌ی بحرانی

شرایط رقابتی؛ چرا همگام‌سازی لازم است؟

دستور ساده‌ی `counter++` در سطح ماشین یک عمل واحد نیست و از سه گام تشکیل می‌شود: خواندن مقدار از حافظه (load)، افزودن یک واحد (add) و نوشتن نتیجه (store). اگر دو نخ این سه گام را به‌صورت درهم‌تنیده اجرا کنند، یکی از به‌روزرسانی‌ها از بین می‌رود. جدول ۱ این وضعیت را برای مقدار اولیه‌ی `counter = ۵` نشان می‌دهد؛ پس از اجرای دو افزایش انتظار می‌رود مقدار ۷ باشد، اما نتیجه ۶ است.

جدول ۱ – نمونه‌ی درهم‌تنیدگی دو نخ که به‌روزرسانی را از بین می‌برد (مقدار اولیه ۵)

گام	نخ ۱	نخ ۲	مقدار حافظه	ثبات‌ها
۱	R۱ = counter	—	۵	R۱=۵
۲	—	R۲ = counter	۵	R۱=۵, R۲=۵
۳	R۱ = R۱ + ۱	—	۵	R۱=۶, R۲=۵
۴	—	R۲ = R۲ + ۱	۵	R۱=۶, R۲=۶
۵	counter = R۱	—	۶	—
۶	—	counter = R۲	۶ (انتظار می‌رفت ۷)	—

سه شرط یک راه‌حل درست

هر راه‌حل برای مسئله‌ی ناحیه‌ی بحرانی باید سه شرط کلاسیک را برآورده کند. این سه شرط معیار اصلی مقایسه در بخش‌های بعدی هستند:

- **انحصار متقابل** (Mutual Exclusion): اگر فرایندی در ناحیه‌ی بحرانی باشد، هیچ فرایند دیگری حق ورود به ناحیه‌ی بحرانی خود را ندارد.
- **پیشرفت** (Progress): اگر ناحیه‌ی بحرانی خالی باشد و فرایندهایی بخواهند وارد شوند، تصمیم برای انتخاب فرایند بعدی نباید بی‌نهایت به تأخیر بیفتد و فقط فرایندهایی در آن دخیل‌اند که در بخش باقی‌مانده نیستند.
- **انتظار محدود** (Bounded Waiting): از لحظه‌ای که فرایندی درخواست ورود می‌دهد تا لحظه‌ی پذیرفته‌شدن، تعداد دفعاتی که دیگران اجازه‌ی ورود می‌گیرند باید محدود باشد؛ یعنی گرسنگی (Starvation) رخ ندهد.

افزون بر این‌ها فرض می‌کنیم هر فرایند با سرعتی نامعلوم و ناصفر اجرا می‌شود و هیچ فرضی درباره‌ی ترتیب زمان‌بندی نداریم. برای رسیدن به این هدف سه گروه ابزار وجود دارد: الگوریتم‌های نرم‌افزاری صرف (دکر و پترسون)، ابزارهای سطح سیستم‌عامل (سمافور) و ساختارهای سطح زبان (مانیتور). مسئله‌ی خوانندگان و نویسندگان نیز یک الگوی همگام‌سازی است که با همین ابزارها حل می‌شود.

معیارهای مقایسه

در این مقاله هر الگوریتم را بر پایه‌ی معیارهای زیر می‌سنجیم: (۱) انحصار متقابل، (۲) پیشرفت، (۳) انتظار محدود، (۴) انتظار مشغول (Busy Waiting) و مصرف CPU، (۵) تعداد فرایندهای پشتیبانی‌شده و مقیاس‌پذیری، (۶) نیاز به پشتیبانی سخت‌افزار یا سیستم‌عامل، (۷) امکان بن‌بست و گرسنگی، (۸) پیچیدگی و اندازه‌ی کد، (۹) سهولت استفاده و صحت‌یابی، و (۱۰) کاربرد عملی.

جدول ۲ — نقشه‌ی پنج راهکار مورد بررسی

راهکار	سطح	سال / ابداع‌کننده	نوع	ایده‌ی اصلی
دکر	نرم‌افزاری	۱۹۶۵ · دکر (منتشرشده توسط دایکسترا)	الگوریتم، ۲ فرایند	پرچم + نوبت، عقب‌نشینی
پترسون	نرم‌افزاری	۱۹۸۱ · پترسون	الگوریتم، ۲ فرایند	پرچم + قربانی (نوبت)
سمافور	ابزار سیستم‌عامل	۱۹۶۵ · دایکسترا	ابزار همگام‌سازی	عدد صحیح + V / P + صف
مانیتور	ساختار زبان	۱۹۷۴ · هور	نوع داده‌ی انتزاعی	قفل + متغیرهای شرطی
خوانندگان–نویسندگان	مسئله / الگو	۱۹۷۱ · کورتوا، هیمنس، پارناس	مسئله‌ی همگام‌سازی	خواندن مشترک، نوشتن انحصاری

۲. الگوریتم دکر

تاریخچه و ایده

الگوریتم دکر نخستین راه‌حل نرم‌افزاری درست برای مسئله‌ی انحصار متقابل میان دو فرایند است. این الگوریتم را ریاضی‌دان هلندی تئودروس دکر (Dekker .J .T) در سال ۱۹۶۰ ابداع کرد و ادسخر دایکسترا آن را در سال ۱۹۶۵ منتشر و به حالت N فرایند تعمیم داد. مزیت تاریخی آن این است که بدون هیچ دستور سخت‌افزاری ویژه (مانند Test-and-Set) و فقط با خواندن و نوشتن متغیرهای مشترک کار می‌کند.

ایده‌ی اصلی از ترکیب دو مکانیزم ساخته می‌شود. نخست، هر فرایند با یک پرچم (wants[i]) اعلام می‌کند که می‌خواهد وارد شود. دوم، متغیر turn مشخص می‌کند در صورت تعارض (هر دو فرایند پرچم خود را بالا برده باشند) کدام فرایند اولویت دارد. فرایندی که اولویت ندارد پرچم خود را موقتاً پایین می‌آورد (**عقب‌نشینی** یا Back-off)، منتظر می‌ماند تا نوبتش شود و دوباره پرچم را بالا می‌برد. این عقب‌نشینی همان چیزی است که الگوریتم را از راه‌حل‌های ساده‌ی «فقط پرچم» که دچار بن‌بست می‌شوند متمایز می‌کند.

شبه‌کد

شبه‌کد ۱ — شبه‌کد الگوریتم دکر برای فرایند i

```
shared: boolean wants[۲] = {false, false}
        int turn = ۰                    // ۰ or ۱

process P(i):                           // j = ۱ - i
  loop forever
    wants[i] <- true                    // announce interest
    while wants[j] do
      if turn != i then                 // the other has priority
        wants[i] <- false               // back off
        wait until turn == i
        wants[i] <- true
    end while
    ----- CRITICAL SECTION -----
    turn <- j                           // give priority away
    wants[i] <- false
    ----- REMAINDER SECTION -----
  end loop
```

پیاده‌سازی به زبان C

کد زیر با متغیرهای اتمیک استاندارد C۱۱ نوشته شده است. استفاده از atomic_int (با ترتیب‌دهی پیش‌فرض seq_cst) ضروری است؛ زیرا روی پردازنده‌های امروزی بدون سد حافظه (Memory Barrier) ترتیب خواندن و نوشتن توسط

پردازنده یا کامپایلر تغییر می‌کند و الگوریتم خراب می‌شود. این کد با دو نخ و میلیون‌ها ورود به ناحیه‌ی بحرانی آزموده شد و شمارنده‌ی مشترک همیشه مقدار درست را داشت.

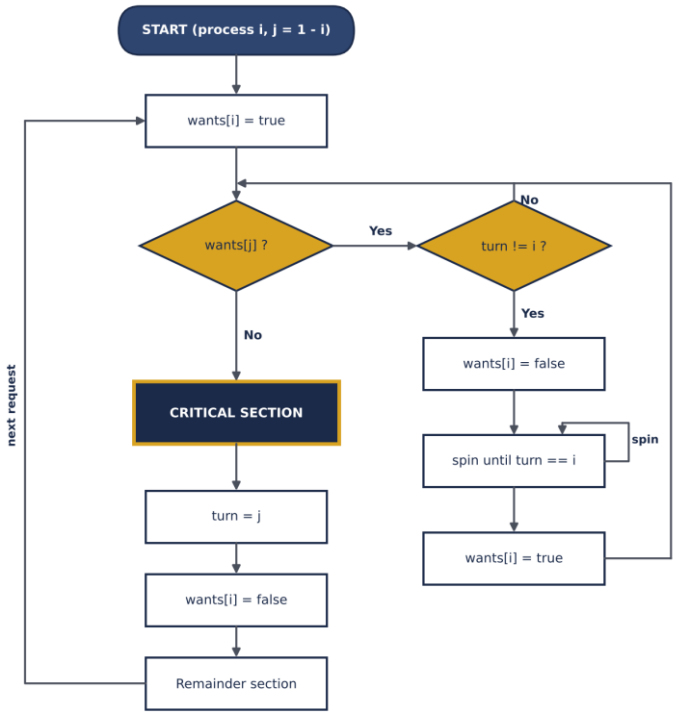
کد ۱ — پیاده‌سازی دکر (C11)

```
#include <stdatomic.h>

atomic_int wants[۲];          /* wants[i] = ۱ : process i wants to enter */
atomic_int turn = ۰;          /* who has priority when both want to enter */

void dekker_lock(int i) {
    int j = ۱ - i;
    atomic_store(&wants[i], ۱);
    while (atomic_load(&wants[j])) {          /* the other one wants too */
        if (atomic_load(&turn) != i) {        /* not my turn: back off   */
            atomic_store(&wants[i], ۰);
            while (atomic_load(&turn) != i)   /* busy wait for my turn   */
                ;
            atomic_store(&wants[i], ۱);
        }
    }
}

void dekker_unlock(int i) {
    atomic_store(&turn, ۱ - i);               /* hand priority over      */
    atomic_store(&wants[i], ۰);
}
```



شکل ۲ – فلوچارت الگوریتم دکر (فرایند i)

تحلیل صحت

- **انحصار متقابل:** برای ورود به ناحیه‌ی بحرانی، فرایند i باید `wants[j] = false` را ببیند. اما فرایند j پرچم خود را پیش از بررسی `wants[i]` بالا برده است. پس دست‌کم یکی از دو فرایند پرچم دیگری را بالا می‌بیند و هر دو هم‌زمان وارد نمی‌شوند.
- **پیشرفت:** اگر هر دو بخواهند وارد شوند، `turn` یکی را برنده می‌کند؛ دیگری عقب‌نشینی می‌کند و برنده می‌تواند ادامه دهد. بن‌بست رخ نمی‌دهد.
- **انتظار محدود:** فرایند خارج‌شونده `turn` را به رقیب می‌دهد؛ پس رقیب حداکثر یک‌بار منتظر می‌ماند و بیشتر از یک‌بار پیش نمی‌افتد.

مزایا و معایب

جنبه	مزایا	معایب
سخت‌افزار	نیازی به دستور ویژه‌ی سخت‌افزاری نیست	Needs sequentially consistent memory (barriers)
CPU	ساده و با حافظه‌ی ناچیز (۳ متغیر)	انتظار مشغول چرخه‌های CPU را هدر می‌دهد
فرایندها	برای ۲ فرایند درست و بدون گرسنگی	فقط ۲ فرایند؛ نسخه‌ی N فرایندی پیچیده است
وضوح	از نظر تاریخی مهم	درک و اثبات دشوار؛ در عمل کم‌کاربرد

۳. الگوریتم پترسون

تاریخچه و ایده

گری پترسون (Peterson .Gary L) در سال ۱۹۸۱ در مقاله‌ای کوتاه با عنوان «افسانه‌هایی درباره‌ی مسئله‌ی انحصار متقابل» الگوریتمی بسیار ساده‌تر از الگوریتم دکر ارائه کرد. این الگوریتم نیز فقط به خواندن و نوشتن متغیرهای مشترک نیاز دارد، اما منطق آن کوتاه‌تر و اثبات صحتش آسان‌تر است و به همین دلیل در کتاب‌های درسی سیستم‌عامل به‌عنوان الگوریتم نرم‌افزاری نمونه آموزش داده می‌شود.

ایده این است که هر فرایند ابتدا علاقه‌ی خود را اعلام می‌کند (flag[i] = true) و سپس **نوبت را به رقیب می‌دهد** (turn = j). فرایند تنها زمانی منتظر می‌ماند که رقیب هم علاقه‌مند باشد و نوبت هم نزد رقیب باشد. اگر هر دو هم‌زمان برسند، آخرین فرایندی که turn را می‌نویسد، نوبت را به دیگری داده است و منتظر می‌ماند؛ بنابراین همیشه دقیقاً یکی عبور می‌کند.

شبه‌کد

شبه‌کد ۲ — شبه‌کد الگوریتم پترسون

```
shared: boolean flag[۲] = {false, false}
        int turn                    // the process that must wait

process P(i):                       // j = ۱ - i
  loop forever
    flag[i] <- true                 // I am interested
    turn    <- j                    // let the other go first
    while flag[j] and turn = j do
      ;                             // busy wait
    ----- CRITICAL SECTION -----
    flag[i] <- false                // I am done
    ----- REMAINDER SECTION -----
  end loop
```

پیاده‌سازی به زبان C

پیاده‌سازی پترسون تنها دوازده خط کد مؤثر دارد. مانند دکر باید از متغیرهای اتمیک C۱۱ استفاده شود. ترتیب دو دستور نوشتن در بخش ورود (flag[i] سپس turn) بسیار مهم است؛ اگر جابه‌جا شوند، انحصار متقابل نقض می‌شود.

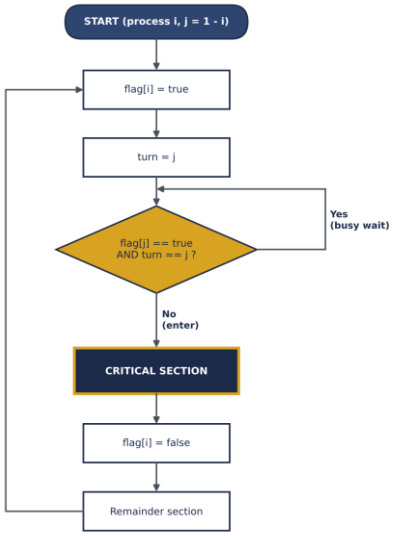
کد ۲ — پیاده‌سازی پترسون (C11)

```
#include <stdatomic.h>

atomic_int flag[۲];          /* flag[i] = ۱ : process i is interested */
atomic_int turn = ۰;         /* the process that must wait            */

void peterson_lock(int i) {
    int j = ۱ - i;
    atomic_store(&flag[i], ۱);       /* I am interested               */
    atomic_store(&turn, j);          /* politely give the turn away   */
    while (atomic_load(&flag[j]) && atomic_load(&turn) == j)
        ;                            /* busy wait                     */
}

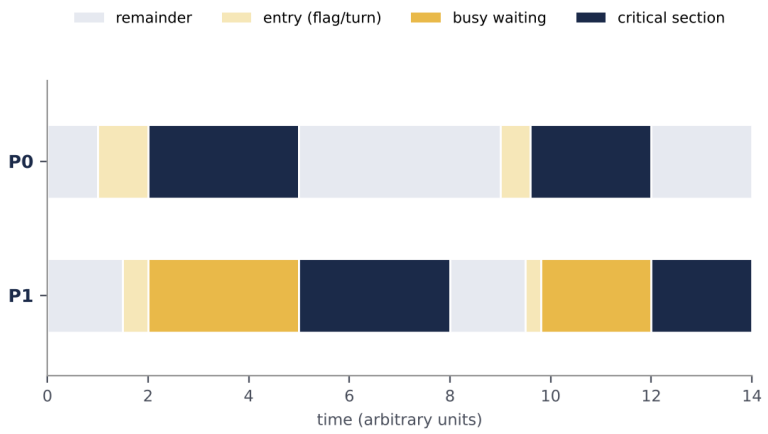
void peterson_unlock(int i) {
    atomic_store(&flag[i], ۰);
}
```



شکل ۳ — فلوچارت الگوریتم پترسون (فرایند i)

نمودار زمانی اجرا

شکل ۴ یک اجرای نمونه را نشان می‌دهد. فرایند P۰ در زمان ۱ و P۱ در زمان ۱٫۵ درخواست ورود می‌دهند. P۰ زودتر می‌رسد و ناحیه‌ی بحرانی را می‌گیرد؛ P۱ تا زمان ۵ با انتظار مشغول می‌ماند. در دور دوم، هر دو تقریباً هم‌زمان می‌رسند؛ چون P۱ آخرین نویسنده‌ی `turn` است نوبت را به P۰ داده و منتظر می‌ماند.



شکل ۴ — نمودار زمانی دو فرایند تحت الگوریتم پترسون

تحلیل صحت

- **انحصار متقابل:** فرض کنید هر دو در ناحیه‌ی بحرانی باشند. هر دو باید شرط حلقه را نادرست دیده باشند، یعنی P۰ باید `flag[۱]=false` یا `turn=۰` ببیند و P۱ نیز `flag[۰]=false` یا `turn=۱`. چون هر دو پرچم خود را پیش از نوشتن `turn` بالا برده‌اند، فقط حالتی باقی می‌ماند که `turn` هم‌زمان ۰ و ۱ باشد که ناممکن است.
- **پیشرفت:** فرایندی که در بخش باقی‌مانده است پرچمش پایین است و دیگری را مسدود نمی‌کند.
- **انتظار محدود:** پس از خروج، پرچم پایین می‌آید؛ اگر رقیب بلافاصله دوباره درخواست دهد، `turn` را به فرایند منتظر می‌دهد؛ پس انتظار حداکثر یک‌بار است.

> **هشدار عملی**
>
> روی پردازنده‌های چندهسته‌ای بدون سد حافظه (یا با متغیرهای ساده‌ی volatile) الگوریتم پترسون ممکن است خراب شود. همچنین اگر نخ دارای ناحیه‌ی بحرانی پیش از پایان کار از CPU خارج شود، نخ رقیب تمام برش زمانی خود را به انتظار مشغول هدر می‌دهد؛ ما این پدیده را در آزمایش خود روی یک هسته‌ی CPU مشاهده کردیم.

مزایا و معایب

جنبه	مزایا	معایب
سادگی	بسیار کوتاه و اثبات صحت آسان	محدود به ۲ فرایند
انصاف	انتظار محدود سخت‌گیرانه (حداکثر ۱ نوبت)	انتظار مشغول CPU را هدر می‌دهد
سخت‌افزار	فقط خواندن و نوشتن اتمیک	روی CPUهای امروزی بدون سد حافظه خراب می‌شود
N فرایند	تعمیم با قفل فیلتر وجود دارد	قفل فیلتر به N-۱ سطح و N-۱۲ متغیر نیاز دارد

۴. سمافور

تعریف و ایده

سمافور را ادسخر دایکسترا در سال ۱۹۶۵ معرفی کرد. سمافور یک متغیر صحیح غیرمنفی (در پیاده‌سازی بدون انتظار مشغول، گاه منفی) است که فقط از طریق دو عمل **اتمیک** تغییر می‌کند: `wait(S)` (در متن اصلی P از Proberen، یعنی «آزمودن») و `signal(S)` (در متن اصلی V از Verhogen، یعنی «افزایش»). عمل wait مقدار را کم می‌کند و اگر منبعی باقی نمانده باشد فرایند را مسدود می‌کند؛ عمل signal مقدار را زیاد می‌کند و در صورت وجود فرایند منتظر یکی را بیدار می‌کند. برخلاف دکر و پترسون، فرایند منتظر در این روش CPU را آزاد می‌کند و می‌خوابد.

انواع سمافور

- **سمافور دودویی** (mutex / Binary): مقدار فقط ۰ یا ۱؛ برای انحصار متقابل.
- **سمافور شمارشی** (Counting): مقدار هر عدد صحیح؛ برای مدیریت N نمونه از یک منبع (مثلاً N اسلات خالی بافر).
- **سمافور قوی و ضعیف** (Weak / Strong): در سمافور قوی صف انتظار FIFO است و انتظار محدود تضمین می‌شود؛ در سمافور ضعیف ترتیب بیداری نامشخص است و گرسنگی ممکن است.

شبه‌کد

شبه‌کد ۳ — پیاده‌سازی سمافور با صف انتظار (بدون انتظار مشغول)

```
type semaphore = record
    int  value
    list queue                  // processes blocked on this semaphore
end record

wait(S):                        // executed atomically
  S.value <- S.value - ۱
  if S.value < ۰ then
    append this process to S.queue
    block()                     // sleep, CPU is released

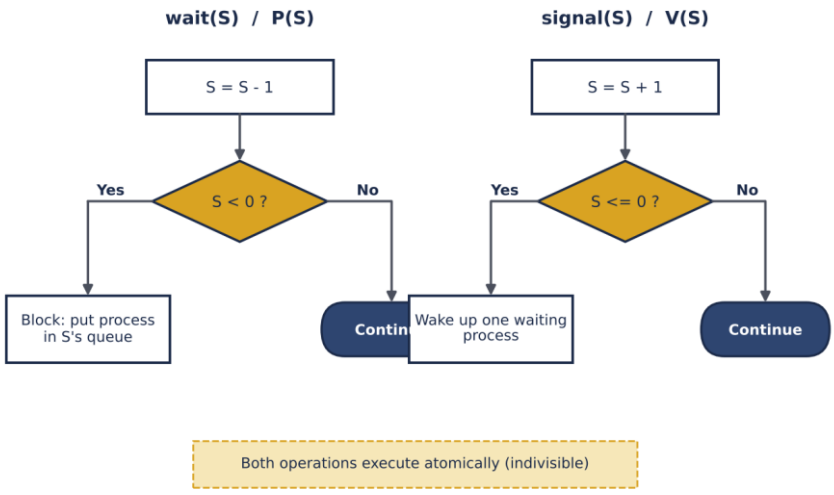
signal(S):                      // executed atomically
  S.value <- S.value + ۱
  if S.value <= ۰ then
    remove a process Q from S.queue
    wakeup(Q)
```

شبه‌کد ۴ — انحصار متقابل و مسئله‌ی تولیدکننده–مصرف‌کننده

```
// (a) mutual exclusion with a binary semaphore
semaphore mutex = ۱
process P(i):
  loop forever
    wait(mutex)
    ----- CRITICAL SECTION -----
    signal(mutex)
    ----- REMAINDER SECTION -----

// (b) bounded buffer with capacity N
semaphore empty = N, full = ۰, mutex = ۱

producer:                    consumer:
  loop forever                 loop forever
    item <- produce()            wait(full)
    wait(empty)                  wait(mutex)
    wait(mutex)                  item <- buffer.remove()
    buffer.insert(item)          signal(mutex)
    signal(mutex)                signal(empty)
    signal(full)                 consume(item)
```



شکل ۵ — عملیات wait و signal در سمافور با صف انتظار

پیاده‌سازی به زبان C

کد زیر مسئله‌ی بافر محدود را با سمافورهای POSIX حل می‌کند. سه سمافور نقش‌های متفاوتی دارند: `empty` تعداد اسلات‌های خالی را می‌شمارد (تولیدکننده روی آن منتظر می‌ماند)، `full` تعداد اسلات‌های پر را می‌شمارد (مصرف‌کننده روی آن منتظر می‌ماند) و `mutex` دسترسی به شاخص‌های بافر را منحصر می‌کند. در آزمایش، یک تولیدکننده ۱۰۰٬۰۰۰ عدد تولید و یک مصرف‌کننده همه را مصرف کرد و مجموع دقیقاً درست بود.

کد ۳ — بافر محدود با سمافور POSIX

```
#include <semaphore.h>

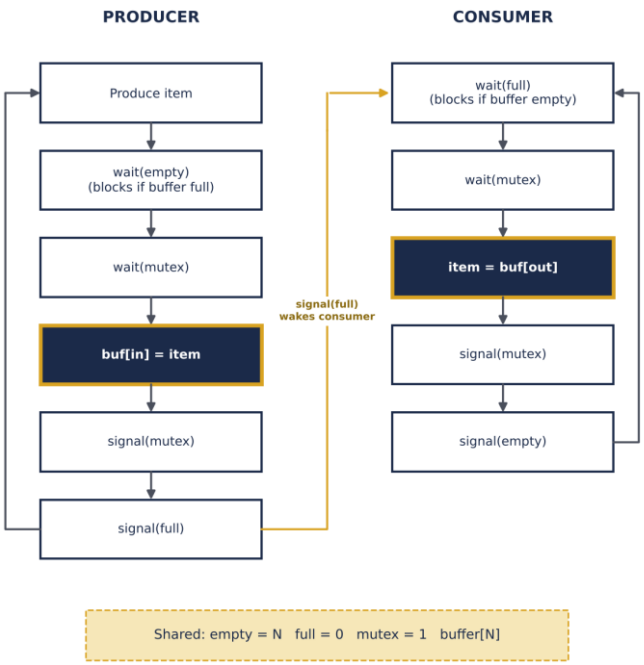
#define N ۸                    /* buffer capacity */
int buf[N], in = ۰, out = ۰;

sem_t empty_slots;              /* init: N  (free slots)  */
sem_t full_slots;               /* init: ۰  (filled slots) */
sem_t mutex;                    /* init: ۱  (binary semaphore) */

void init(void) {
    sem_init(&empty_slots, ۰, N);
    sem_init(&full_slots,  ۰, ۰);
    sem_init(&mutex,       ۰, ۱);
}

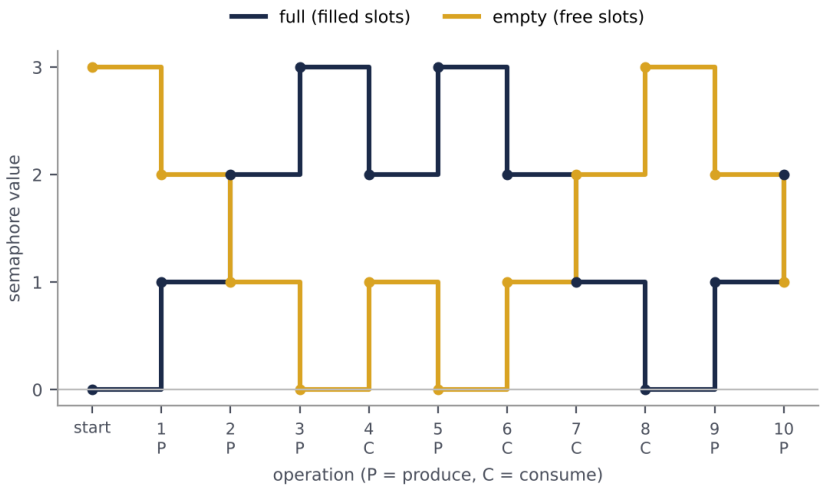
void produce(int item) {
    sem_wait(&empty_slots);     /* P(empty) */
    sem_wait(&mutex);           /* P(mutex) */
    buf[in] = item;  in = (in + ۱) % N;
    sem_post(&mutex);           /* V(mutex) */
    sem_post(&full_slots);      /* V(full)  */
}

int consume(void) {
    int item;
    sem_wait(&full_slots);      /* P(full)  */
    sem_wait(&mutex);           /* P(mutex) */
    item = buf[out];  out = (out + ۱) % N;
    sem_post(&mutex);           /* V(mutex) */
    sem_post(&empty_slots);     /* V(empty) */
    return item;
}
```



شکل ۶ — فلوچارت تولیدکننده و مصرف‌کننده با سمافور

شکل ۷ تغییر مقدار سمافورها را برای بافری با ظرفیت ۳ و ده عملیات متوالی نشان می‌دهد. مجموع `empty + full` همیشه برابر ۳ است. در گام ۳ بافر پر می‌شود (`empty = ۰`)؛ اگر تولیدکننده‌ای در این لحظه عمل چهارم تولید را انجام دهد روی `wait(empty)` مسدود می‌شود تا مصرف‌کننده در گام ۴ یک اسلات آزاد کند.



شکل ۷ — تغییر مقدار سمافورهای empty و full (ظرفیت بافر ۳)

تحلیل و خطرات

- **انحصار متقابل:** با مقدار اولیه‌ی ۱ و جفت‌های wait/signal، حداکثر یک فرایند از wait عبور می‌کند.

- **بن‌بست:** اگر در تولیدکننده ترتیب `wait(mutex)` و `wait(empty)` عوض شود، ممکن است تولیدکننده با نگه داشتن mutex روی `empty` بخوابد و مصرف‌کننده هرگز نتواند mutex را بگیرد.
- **خطای برنامه‌نویس:** فراموش کردن `signal` یا اجرای اضافی `wait` به بن‌بست یا نقض انحصار می‌انجامد؛ کامپایلر هیچ کمکی نمی‌کند، زیرا فراخوانی‌ها در کد پراکنده‌اند.
- **وارونگی اولویت:** فرایند کم‌اولویت که سمافور را نگه داشته می‌تواند فرایند پراولویت را معطل کند (مانند ماجرای Mars Pathfinder در ۱۹۹۷).

جنبه	مزایا	معایب
CPU	مسدودسازی: فرایند منتظر می‌خوابد و CPU هدر نمی‌رود	هزینه‌ی تعویض متن برای نواحی بحرانی بسیار کوتاه
عمومیت	انحصار متقابل و ارسال سیگنال؛ N فرایند	سطح پایین؛ مستعد خطا (ترتیب، فراموشی signal)
انصاف	سمافور قوی انتظار محدود را تضمین می‌کند	سمافور ضعیف ممکن است گرسنگی ایجاد کند
پشتیبانی	در همه‌ی سیستم‌عامل‌های اصلی موجود است (POSIX، Windows)	به پشتیبانی OS و سخت‌افزار اتمیک نیاز دارد

۵. مانیتور

تعریف و ایده

با وجود قدرت سمافور، برنامه‌نویسی با آن مستعد خطاست. تونی هور (Hoare .R .A .C) در سال ۱۹۷۴ و بر پایه‌ی کار برینچ هانسن، **مانیتور** را به‌عنوان یک ساختار سطح بالا معرفی کرد. مانیتور یک نوع داده‌ی انتزاعی است که داده‌های مشترک، رویه‌هایی که روی آن‌ها کار می‌کنند و قفل همگام‌سازی را در یک واحد بسته‌بندی می‌کند. قاعده‌ی اصلی: **در هر لحظه فقط یک نخ می‌تواند داخل مانیتور فعال باشد**. قفل به‌طور خودکار هنگام ورود به هر رویه گرفته و هنگام خروج آزاد می‌شود؛ پس برنامه‌نویس نمی‌تواند آن را فراموش کند.

برای همگام‌سازی شرطی (مثلاً «صبر کن تا بافر پر نباشد») مانیتور از **متغیرهای شرطی** (Condition Variables) استفاده می‌کند. عمل `wait(c)` نخ را در صف شرط می‌خواباند و در همان لحظه قفل مانیتور را آزاد می‌کند؛ عمل `signal(c)` یک نخ منتظر را بیدار می‌کند. برخلاف سمافور، متغیر شرطی حافظه ندارد: اگر signal زده شود و کسی منتظر نباشد، اثر آن از بین می‌رود.

دو سبک سیگنال: هور و مزا

ویژگی	سبک هور	سبک مزا (Mesa)
signal	فرستنده‌ی signal بلافاصله مانیتور را به نخ بیدارشده می‌دهد	فرستنده ادامه می‌دهد؛ نخ بیدارشده فقط آماده می‌شود
آزمون شرط	`if` کافی است (شرط هنوز برقرار است)	باید از `while` استفاده کرد (شرط ممکن است تغییر کرده باشد)
هزینه	تعویض متن بیشتر	ارزان‌تر؛ در عمل استفاده می‌شود
پیاده‌سازی‌ها	کتاب‌های درسی و نظریه	Java، C#، نخ‌های POSIX، Python

شبه‌کد

شبه‌کد ۵ — مانیتور بافر محدود

```
monitor BoundedBuffer
  var buf[N], in = ۰, out = ۰, count = ۰
  condition notFull, notEmpty

  procedure put(item)
    while count = N do wait(notFull)      // Mesa style: re-test
    buf[in] <- item;  in <- (in + ۱) mod N;  count <- count + ۱
    signal(notEmpty)

  procedure get() returns item
    while count = ۰ do wait(notEmpty)
    item <- buf[out]; out <- (out + ۱) mod N; count <- count - ۱
    signal(notFull)
    return item
end monitor
// the monitor lock is acquired on entry and released on exit
// of every procedure (and while waiting) automatically
```

پیاده‌سازی در C و Java

زبان C مانیتور ندارد، اما می‌توان آن را با یک `pthread_mutex_t` و چند `pthread_cond_t` ساخت. در زبان Java هر شیء به‌طور ذاتی یک مانیتور است: کلیدواژه‌ی `synchronized` قفل را می‌گیرد و `wait()` / `notifyAll()` نقش متغیر شرطی را دارند. نسخه‌ی C این مقاله با یک تولیدکننده و یک مصرف‌کننده و ۱۰۰٬۰۰۰ عنصر آزموده شد؛ نسخه‌ی Java ساختاری استاندارد دارد.

کد ۴ — مانیتور بافر محدود با pthread

```
#include <pthread.h>

#define N ۸
/* ---- monitor "BoundedBuffer" : all data + lock + conditions ---- */
typedef struct {
    int buf[N];
    int in, out, count;
    pthread_mutex_t lock;         /* the monitor's implicit mutex  */
    pthread_cond_t  not_full;     /* condition variable            */
    pthread_cond_t  not_empty;    /* condition variable            */
} Monitor;

Monitor m = { .lock = PTHREAD_MUTEX_INITIALIZER,
              .not_full  = PTHREAD_COND_INITIALIZER,
              .not_empty = PTHREAD_COND_INITIALIZER };

void mon_put(int item) {                  /* monitor procedure */
    pthread_mutex_lock(&m.lock);          /* enter monitor     */
    while (m.count == N)                  /* Mesa style: while */
        pthread_cond_wait(&m.not_full, &m.lock);
```

```
    m.buf[m.in] = item;  m.in = (m.in + ۱) % N;  m.count++;
    pthread_cond_signal(&m.not_empty);
    pthread_mutex_unlock(&m.lock);          /* leave monitor     */
}

int mon_get(void) {                         /* monitor procedure */
    pthread_mutex_lock(&m.lock);
    while (m.count == ۰)
        pthread_cond_wait(&m.not_empty, &m.lock);
    int item = m.buf[m.out];  m.out = (m.out + ۱) % N;  m.count--;
    pthread_cond_signal(&m.not_full);
    pthread_mutex_unlock(&m.lock);
    return item;
}
```

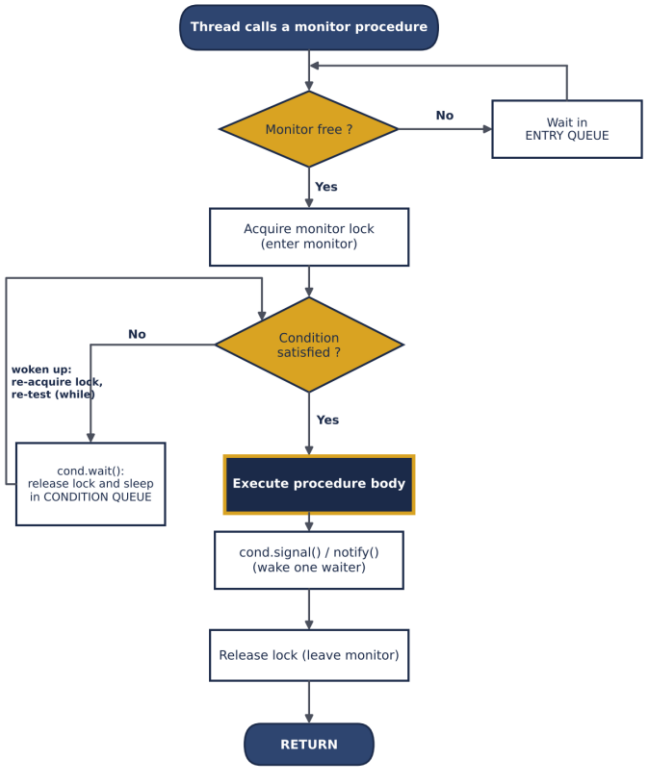
کد ۵ — مانیتور بافر محدود در Java

```
public class BoundedBuffer {
    private final int[] buf;
    private int in = ۰, out = ۰, count = ۰;

    public BoundedBuffer(int capacity) { buf = new int[capacity]; }

    public synchronized void put(int item) throws InterruptedException {
        while (count == buf.length)          // Mesa semantics: always re-test
            wait();
        buf[in] = item;  in = (in + ۱) % buf.length;  count++;
        notifyAll();                         // wake up waiting threads
    }

    public synchronized int get() throws InterruptedException {
        while (count == ۰)
            wait();
        int item = buf[out];  out = (out + ۱) % buf.length;  count--;
        notifyAll();
        return item;
    }
}
```



شکل ۸ – فلوچارت اجرای یک رویه‌ی مانیتور

تحلیل

- **انحصار متقابل:** به‌صورت ساختاری تضمین می‌شود؛ هیچ کد کاربر برای قفل‌کردن لازم نیست.
- **صف‌ها:** نخ‌ها دو نوع صف دارند: صف ورود به مانیتور و صف هر متغیر شرطی. انتظار محدود به سیاست زمان‌بندی FIFO بودن صف‌ها) وابسته است.
- **بن‌بست:** فراخوانی تو در توی مانیتورها (Nested Monitor Call) یا `wait` بدون `signal` متناظر می‌تواند به بن‌بست برسد.
- **تبادل نشانه‌ها:** در سبک Mesa حلقه‌ی `while` الزامی است؛ استفاده از `if` به بیدارشدن‌های کاذب (Spurious Wakeup) و خطا می‌انجامد.

جنبه	مزایا	معایب
ایمنی	قفل را زبان مدیریت می‌کند؛ سوءاستفاده دشوار	نیازمند پشتیبانی زبان یا زمان‌اجرا (در C ساده نیست)
ماژولار بودن	داده و همگام‌سازی در یک واحد کپسوله می‌شوند	فراخوانی‌های تو در تو می‌توانند بن‌بست بسازند
شرط‌ها	متغیرهای شرطی انتظارهای پیچیده را شفاف بیان می‌کنند	معناشناسی signal (هور در برابر Mesa) مبتدیان را گیج می‌کند
کارایی	مسدودسازی، بدون انتظار مشغول	قفل درشت‌دانه: در کل مانیتور فقط یک نخ

۶. خوانندگان و نویسندگان

صورت مسئله

مسئله‌ی خوانندگان و نویسندگان را کورتوا، هیمنس و پارناس در سال ۱۹۷۱ مطرح کردند. چند فرایند «خواننده» فقط داده‌ی مشترک را می‌خوانند و چند فرایند «نویسنده» آن را تغییر می‌دهند. قاعده‌ها: (۱) چند خواننده می‌توانند هم‌زمان بخوانند؛ (۲) نویسنده باید دسترسی انحصاری داشته باشد، یعنی هنگام نوشتن نه خواننده‌ای باشد و نه نویسنده‌ی دیگر. این مسئله مدل پایگاه‌داده‌ها، کش‌ها و فایل‌سیستم‌هاست که در آن‌ها خواندن بسیار بیشتر از نوشتن رخ می‌دهد و انحصار کامل (مثل mutex) کارایی را بی‌دلیل کم می‌کند.

> **نکته**
>
> خوانندگان–نویسندگان در اصل یک «مسئله‌ی همگام‌سازی» است، نه یک ابزار مانند سمافور؛ اما چون راه‌حل‌های استاندارد آن در همه‌ی کتاب‌ها ارائه می‌شود و در عمل به‌صورت قفل خواندن/نوشتن (RW-Lock) در کتابخانه‌ها وجود دارد، در این مقاله هم‌ردیف الگوریتم‌ها مقایسه می‌شود.

سه گونه‌ی مسئله

گونه	قاعده	پیامد
اول (اولویت با خواننده)	هیچ خواننده‌ای منتظر نمی‌ماند مگر نویسنده داده را در اختیار داشته باشد	نویسنده‌ها ممکن است گرسنه شوند
دوم (اولویت با نویسنده)	وقتی نویسنده‌ای منتظر است خواننده‌ی جدید شروع نمی‌کند	خواننده‌ها ممکن است گرسنه شوند
سوم (منصفانه)	درخواست‌ها به ترتیب ورود (دروازه / صف) سرویس می‌شوند	بدون گرسنگی، هم‌روندی خواندن کمی کمتر

شبه‌کد

شبه‌کد ۶ — خوانندگان–نویسندگان، گونه‌ی اول (اولویت با خواننده)

```
shared: semaphore mutex = ۱, rw_mutex = ۱
        int read_count = ۰

writer:                          reader:
  wait(rw_mutex)                   wait(mutex)
  ----- WRITE -----                read_count <- read_count + ۱
  signal(rw_mutex)                 if read_count = ۱ then wait(rw_mutex)
                                   signal(mutex)
                                   ----- READ -----
                                   wait(mutex)
                                   read_count <- read_count - ۱
                                   if read_count = ۰ then signal(rw_mutex)
                                   signal(mutex)
```

شبه‌کد ۷ — گونه‌ی منصفانه با دروازه‌ی turnstile

```
shared: semaphore mutex = 1, rw_mutex = 1, turnstile = 1
        int read_count = 0

writer:                          reader:
  wait(turnstile)                  wait(turnstile)    // pass the gate
  wait(rw_mutex)                   signal(turnstile)
  ----- WRITE -----               (same entry code as before)
  signal(turnstile)                ----- READ -----
  signal(rw_mutex)                 (same exit code as before)
// a waiting writer holds the turnstile, so no new reader can pass it
```

در نسخه‌ی اول، «اولین خواننده» نویسندگان را بیرون می‌اندازد و «آخرین خواننده» آن‌ها را راه می‌دهد؛ شمارنده‌ی `read_count` با `mutex` محافظت می‌شود. در نسخه‌ی منصفانه، نویسنده‌ی منتظر دروازه‌ی `turnstile` را می‌بندد و خواننده‌های جدید پشت آن می‌ایستند؛ پس نویسنده پس از خروج خواننده‌های فعلی حتماً به‌نوبت می‌رسد.

پیاده‌سازی به زبان C

پیاده‌سازی زیر هر دو نسخه را با سمافور POSIX ارائه می‌دهد. در آزمایش، سه خواننده و دو نویسنده به‌طور هم‌زمان اجرا شدند؛ نویسنده‌ها دو عضو یک آرایه را با فاصله‌ی زمانی افزایش می‌دادند و خواننده‌ها بررسی می‌کردند دو عضو همیشه برابرند. هیچ تخلفی در هیچ‌یک از دو نسخه مشاهده نشد.

کد ۶ — خوانندگان–نویسندگان (ساده و منصفانه)

```
#include <semaphore.h>

/* --------- Readers-Writers, first problem (readers preference) --------- */
sem_t rw_mutex;                 /* init 1 : exclusive access to the data   */
sem_t mutex;                    /* init 1 : protects read_count            */
int   read_count = 0;

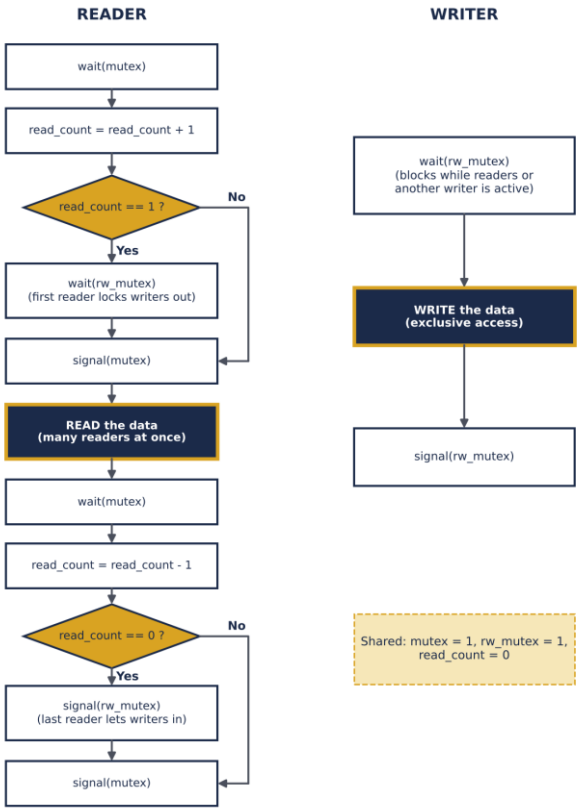
void writer_enter(void) { sem_wait(&rw_mutex); }
void writer_exit (void) { sem_post(&rw_mutex); }

void reader_enter(void) {
    sem_wait(&mutex);
    if (++read_count == 1)      /* first reader locks out the writers      */
        sem_wait(&rw_mutex);
    sem_post(&mutex);
}
void reader_exit(void) {
    sem_wait(&mutex);
    if (--read_count == 0)      /* last reader lets the writers in         */
        sem_post(&rw_mutex);
    sem_post(&mutex);
}
```

```
/* --------- Fair version: a turnstile stops new readers once a writer waits --------- */
sem_t turnstile;                /* init ۱ */

void fair_writer_enter(void) { sem_wait(&turnstile); sem_wait(&rw_mutex); }
void fair_writer_exit (void) { sem_post(&turnstile); sem_post(&rw_mutex); }

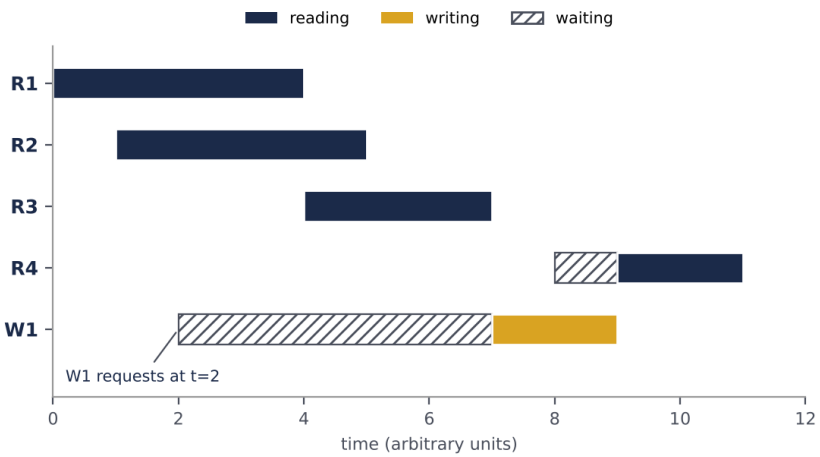
void fair_reader_enter(void) {
    sem_wait(&turnstile);  sem_post(&turnstile);   /* pass through the gate */
    reader_enter();
}
void fair_reader_exit(void) { reader_exit(); }
```



شکل ۹ — فلوچارت خواننده و نویسنده (گونه‌ی اول)

نمودار زمانی و گرسنگی

شکل ۱۰ اجرای گونه‌ی اول را نشان می‌دهد. نویسنده‌ی W۱ در زمان ۲ درخواست می‌دهد اما چون خواننده‌ی R۳ در زمان ۴ هنوز می‌تواند به خوانندگان فعال بپیوندد، W۱ تا زمان ۷ منتظر می‌ماند. اگر خواننده‌های جدید پشت سر هم بیایند، W۱ هرگز نوبت نمی‌گیرد. این همان گرسنگی نویسنده در گونه‌ی اول است.



شکل ۱۰ — نمودار گانت: گرسنگی نویسنده در گونه‌ی اول

تحلیل

- **انحصار متقابل:** نویسنده با `rw_mutex` از همه خارج می‌شود؛ خوانندگان با هم مشترک‌اند ولی با نویسنده نه.
- **هم‌روندی:** تا N خواننده هم‌زمان می‌خوانند؛ این ویژگی در هیچ‌یک از چهار الگوریتم دیگر وجود ندارد.
- **گرسنگی:** گونه‌ی اول نویسنده‌ها را و گونه‌ی دوم خواننده‌ها را گرسنه می‌کند؛ گونه‌ی سوم هر دو را حفظ می‌کند.
- **کاربرد:** `pthread_rwlock_t` در POSIX و `ReentrantReadWriteLock` در Java همین الگو را آماده ارائه می‌دهند.

جنبه	مزایا	معایب
توان عملیاتی	خواندن موازی: توان بالا برای بارهای خواندنی	اگر نوشتن زیاد باشد، mutex ساده سریع‌تر و ساده‌تر است
انصاف	گونه‌ی منصفانه از گرسنگی جلوگیری می‌کند	گونه‌های ساده نویسنده یا خواننده را گرسنه می‌کنند
پیچیدگی	ساخته‌شده از ۲ تا ۳ سمافور و یک شمارنده	حالت‌ها و ظرافت ترتیب بیشتر از قفل ساده

۷. مقایسه‌ی جامع

در این فصل پنج راهکار از دو دیدگاه مقایسه می‌شوند: «صحت و رفتار» (جدول ۳) و «مهندسی و کارایی» (جدول ۴). سپس امتیازدهی کیفی در جدول ۵ و نمودارهای مقایسه‌ای ارائه می‌شود. مقادیر کمّی مانند تعداد خطوط کد از همین کدهای آزموده‌شده‌ی مقاله به‌دست آمده است.

مقایسه‌ی صحت و رفتار

جدول ۳ – مقایسه‌ی ویژگی‌های صحت و رفتار (انحصار متقابل، پیشرفت، انتظار محدود و ...)

پارامتر	دکر	پترسون	سمافور	مانیتور	خوانندگان–نویسندگان
انحصار متقابل	بله	بله	بله	بله (ساختاری)	نویسنده انحصاری؛ خواننده‌ها مشترک
پیشرفت	بله	بله	بله	بله	بله
انتظار محدود	بله (حداکثر ۱ نوبت)	بله (حداکثر ۱ نوبت)	فقط سمافور قوی (FIFO)	وابسته به سیاست صف	گونه‌ی ۱ و ۲ خیر؛ گونه‌ی منصفانه بله
گرسنگی	خیر	خیر	ممکن (سمافور ضعیف)	ممکن (Mesa)	ممکن (نویسنده یا خواننده)
بن‌بست	خیر	خیر	ممکن (ترتیب اشتباه wait)	ممکن (مانیتور تو در تو)	خودبه‌خود خیر؛ با قفل‌های دیگر ممکن
انتظار مشغول	بله	بله	خیر (مسدودسازی)	خیر	خیر
تعداد فرایند	۲	۲	N	N	N خواننده + M نویسنده
هم‌روندی در بخش بحرانی	خیر	خیر	سمافور شمارشی: تا k نخ	خیر (یک نخ)	بله: خواننده‌های هم‌زمان
همگام‌سازی شرطی	خیر	خیر	بله (signal / wait)	بله (متغیر شرطی)	با سمافور یا مانیتور
وارونگی اولویت	ممکن هنگام چرخش	ممکن هنگام چرخش	ممکن	ممکن	ممکن

مقایسه‌ی مهندسی و کارایی

جدول ۴ – مقایسه‌ی پارامترهای مهندسی: سخت‌افزار، CPU، مقیاس‌پذیری، پیچیدگی و کاربرد

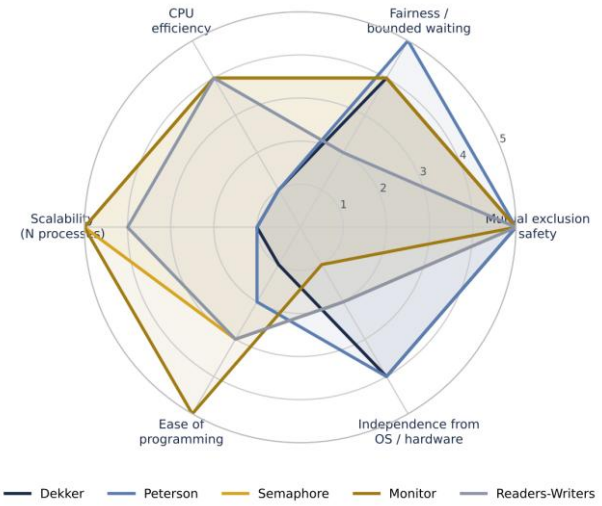
پارامتر	دکر	پترسون	سمافور	مانیتور	خوانندگان–نویسندگان
سطح راهکار	الگوریتم نرم‌افزاری	الگوریتم نرم‌افزاری	ابزار سیستم‌عامل	ساختار زبان	الگوی همگام‌سازی
نیاز پشتیبانی	خواندن/نوشتن اتمیک + سد حافظه	خواندن/نوشتن اتمیک + سد حافظه	زمان‌بند OS + عمل اتمیک	زبان یا کتابخانه‌ی mutex/cond	سمافور / مانیتور / RW-lock
مصرف CPU هنگام انتظار	زیاد	زیاد	کم	کم	کم
مقیاس‌پذیری	ضعیف (۲+N۱ متغیر)	ضعیف (فیلتر: N۲–۱)	خوب (۱ متغیر)	خوب	خوب (۳ متغیر)
خطوط کد (کد این مقاله)	۱۸	۱۲	۲۶	۲۸ (C)	۲۵
متغیرهای همگام‌سازی (۲ فرایند)	۳	۳	۳	۳ (قفل + ۲ شرط)	۳
سهولت صحت‌یابی	دشوار	متوسط (اثبات کوتاه)	متوسط (خطاپذیر)	آسان (ساخت‌یافته)	متوسط
قابلیت اطمینان در چندهسته‌ای	نیازمند سد حافظه	نیازمند سد حافظه	مطمئن	مطمئن	مطمئن
کاربرد امروزی	آموزش و نظریه	آموزش و راستی‌آزمایی	هسته‌ی OS، POSIX، Windows	Java، C#، Python	پایگاه‌داده، کش، فایل‌سیستم

امتیازدهی کیفی

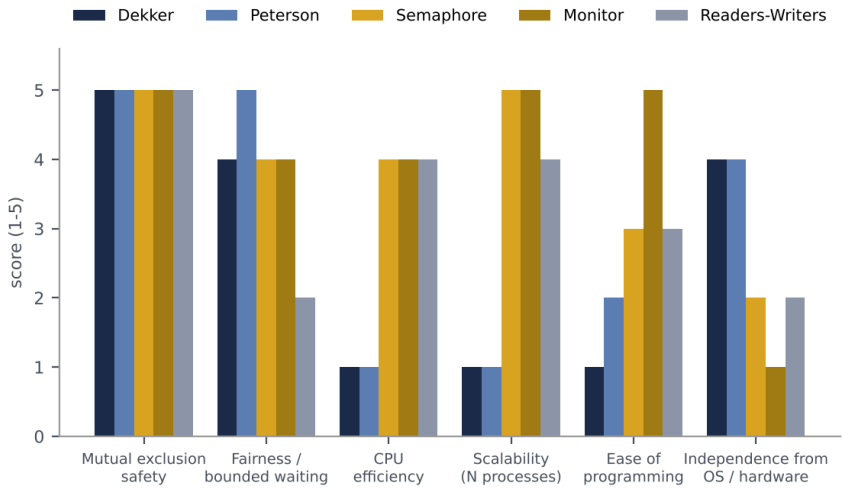
جدول ۵ برای هر معیار امتیازی از ۱ (ضعیف) تا ۵ (عالی) می‌دهد. این امتیازها **ارزیابی کیفی نگارنده** بر پایه‌ی جدول‌های قبل است، نه نتیجه‌ی اندازه‌گیری؛ معیار «استقلال از OS و سخت‌افزار» هرچه بالاتر باشد یعنی راهکار به پشتیبانی کمتری نیاز دارد. وزن همه‌ی معیارها برابر گرفته شده و میانگین در ستون آخر آمده است.

جدول ۵ – امتیاز کیفی راهکارها (۱ تا ۵)

پارامتر	دکر	پترسون	سمافور	مانیتور	خوانندگان–نویسندگان
سلامت انحصار متقابل	۵	۵	۵	۵	۵
انصاف / انتظار محدود	۴	۵	۴	۴	۲
کارایی CPU	۱	۱	۴	۴	۴
مقیاس‌پذیری	۱	۱	۵	۵	۴
سهولت برنامه‌نویسی	۱	۲	۳	۵	۳
استقلال از OS و سخت‌افزار	۴	۴	۲	۱	۲
میانگین	۲٫۶۷	۳٫۰۰	۳٫۸۳	۴٫۰۰	۳٫۳۳

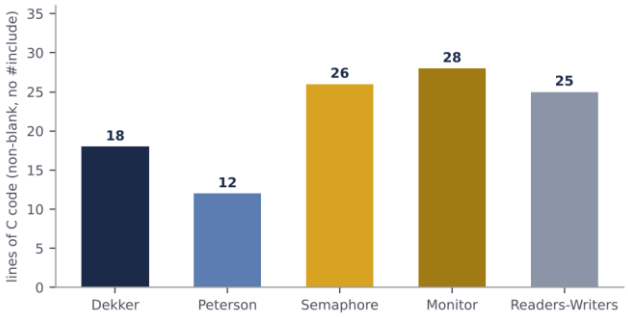


شکل ۱۱ — نمودار راداری مقایسه‌ی شش معیار

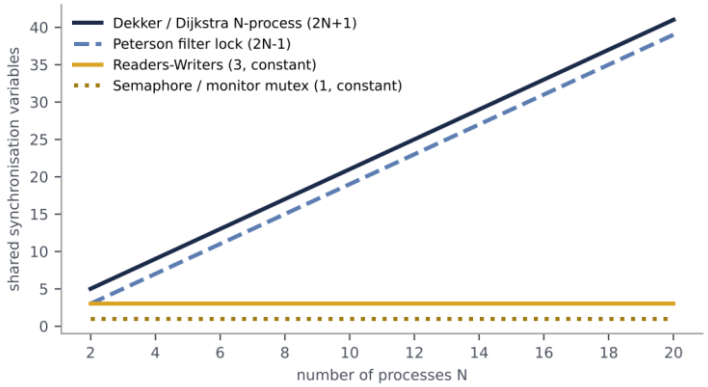


شکل ۱۲ — نمودار میله‌ای گروهی امتیاز هر معیار

نمودار راداری نشان می‌دهد دکر و پترسون فقط در «سلامت» و «استقلال از OS» قوی‌اند، در حالی که سمافور و مانیتور در کارایی و مقیاس‌پذیری برتری دارند. مانیتور بیشترین سهولت برنامه‌نویسی را دارد اما به پشتیبانی زبان وابسته است. گونه‌ی ساده‌ی خوانندگان–نویسندگان در انصاف ضعیف‌تر است ولی هم‌روندی خواندن را فراهم می‌کند.



شکل ۱۳ — اندازه‌ی کد پیاده‌سازی‌ها (خطوط مؤثر C)



شکل ۱۴ — تعداد متغیرهای مشترک همگام‌سازی نسبت به تعداد فرایندها

شکل ۱۴ مقیاس‌پذیری را نشان می‌دهد. تعمیم الگوریتم‌های نرم‌افزاری به N فرایند به تعداد متغیرهای خطی (حدود N۲) نیاز دارد و هر ورود باید همه‌ی آن‌ها را بخواند؛ اما سمافور و مانیتور با تعداد ثابتی متغیر برای هر تعداد فرایند کار می‌کنند و نخ‌ها را به‌جای چرخش به خواب می‌برند.

۸. مثال‌های حل‌شده و نمونه‌سؤال

برای هر یک از پنج الگوریتم یک مثال حل‌شده (ردگیری گام‌به‌گام اجرا) و سه نمونه‌سؤال (دو تستی و یک تشریحی) با پاسخ آمده است. در همه‌ی ردگیری‌ها فرض شده است که زمان‌بند هر بار فقط یک دستور از یکی از فرایندها را اجرا می‌کند.

دکر

مثال ۱: ردگیری ورود هم‌زمان دو فرایند

وضعیت اولیه: `wants = [F, F]` و `turn = ۰`. هر دو فرایند هم‌زمان درخواست ورود می‌دهند. بررسی کنید کدام وارد می‌شود.

گام	فرایند	دستور	wants	turn
۱	P۰	wants[۰] = true	T, F	۰
۲	P۱	wants[۱] = true	T, T	۰
۳	P۰	T=[۱]wants را می‌بیند؛ چون turn = ۰ = i عقب‌نشینی نمی‌کند و حلقه را ادامه می‌دهد	T, T	۰
۴	P۱	T=[۰]wants را می‌بیند؛ turn ≠ ۱ → عقب‌نشینی: wants[۱] = false و چرخش	T, F	۰
۵	P۰	F=[۱]wants را می‌بیند ← از حلقه خارج و وارد ناحیه‌ی بحرانی می‌شود	T, F	۰
۶	P۰	خروج: turn = ۱ و wants[۰] = false	F, F	۱
۷	P۱	turn = ۱ ← wants[۱] = true؛ F=[۰]wants ← وارد ناحیه‌ی بحرانی می‌شود	F, T	۱

نتیجه: P۰ زودتر وارد می‌شود چون `turn = ۰` به او اولویت می‌دهد؛ P۱ عقب می‌نشیند و پس از خروج P۰ به‌نوبت وارد می‌شود. انحصار متقابل و انتظار محدود حفظ شد.

سؤال ۱. در الگوریتم دکر هدف از عقب‌نشینی (`wants[i] = false`) چیست؟

الف) افزایش سرعت اجرا ب) جلوگیری از بن‌بست وقتی هر دو پرچم بالا است و نوبت با رقیب است ج) آزادسازی حافظه د) تعویض نوبت

پاسخ: گزینه‌ی «ب». فرایندی که اولویت ندارد پرچم را پایین می‌آورد تا رقیب بتواند عبور کند.

سؤال ۲. نسخه‌ی اصلی الگوریتم دکر حداکثر برای چند فرایند کار می‌کند؟

الف) ۱ ب) ۲ ج) N د) نامحدود

پاسخ: گزینه‌ی «ب». نسخه‌ی N فرایندی را دایکسترا جداگانه ارائه کرد.

سؤال ۳. اگر در بخش خروج دستور `turn = j` حذف شود، کدام شرط نقض می‌شود؟ توضیح دهید.

پاسخ: انتظار محدود. اگر `turn = ۰` برای همیشه بماند، P۱ در هر تعارض باید عقب بنشیند و P۰ که مرتب درخواست می‌دهد می‌تواند همیشه برنده شود؛ پس P۱ دچار گرسنگی می‌شود. انحصار متقابل همچنان برقرار است.

پترسون

مثال ۲: ردگیری و نقش ترتیب دستورها

وضعیت اولیه: `flag = [F, F]`. P۰ و P۱ درهم‌تنیده اجرا می‌شوند. در شرط حلقه‌ی هر فرایند، `j` رقیب اوست.

گام	فرایند	دستور	flag	turn
۱	P۰	flag[۰] = true	T, F	?
۲	P۱	flag[۱] = true	T, T	?
۳	P۱	turn = ۰ (نوبت را به P۰ می‌دهد)	T, T	۰
۴	P۰	turn = ۱ (نوبت را به P۱ می‌دهد)	T, T	۱
۵	P۰	flag[۱] ∧ turn = ۱ ← درست ← P۰ منتظر می‌ماند	T, T	۱
۶	P۱	flag[۰] ∧ turn = ۰ ← نادرست ← P۱ وارد ناحیه‌ی بحرانی می‌شود	T, T	۱
۷	P۱	خروج: flag[۱] = false	T, F	۱
۸	P۰	flag[۱] = F ← شرط نادرست ← P۰ وارد ناحیه‌ی بحرانی می‌شود	T, F	۱

نتیجه: آخرین فرایندی که `turn` را می‌نویسد (P۰) منتظر می‌ماند و دیگری (P۱) وارد می‌شود.

سؤال ۴. در الگوریتم پترسون اگر دو فرایند هم‌زمان برسند کدام منتظر می‌ماند؟

الف) اولی که پرچم را بالا برد ب) آخرین فرایندی که `turn` را نوشت ج) فرایند با شماره‌ی کمتر د) هر دو

پاسخ: گزینه‌ی «ب».

سؤال ۵. شرط حلقه‌ی انتظار در پترسون کدام است؟

الف) `flag[j] || turn == j` ب) `flag[j] && turn == j` ج) `!flag[j] && turn == i` د) `flag[i] && turn == i`

پاسخ: گزینه‌ی «ب»: فرایند فقط وقتی منتظر می‌ماند که رقیب علاقه‌مند باشد و نوبت هم با رقیب باشد.

سؤال ۶. اگر ترتیب دو دستور ورود را عوض کنیم (ابتدا `turn = j` و بعد `flag[i] = true`) نشان دهید انحصار متقابل نقض می‌شود.

پاسخ: ردگیری: P۰: `turn = ۱`؛ P۱: `turn = ۰`؛ P۱: `flag[۱] = true` و چون `flag[۰] = false` است وارد ناحیه‌ی بحرانی می‌شود؛ سپس P۰: `flag[۰] = true` و چون `turn = ۰` است شرط `turn == ۱` نادرست است و P۰ هم وارد می‌شود. هر دو هم‌زمان در ناحیه‌ی بحرانی‌اند.

سمافور

مثال ۳: بافر محدود با ظرفیت ۲

مقدار اولیه: empty = ۲، full = ۰. سمافورها طبق پیاده‌سازی مسدودکننده ممکن است منفی شوند (قدر مطلق مقدار منفی = تعداد فرایندهای منتظر).

گام	عمل	empty	full	توضیح
۱	تولیدکننده: (empty)wait … (full)signal (عنصر A)	۱	۱	A ذخیره شد
۲	تولیدکننده: عنصر B	۰	۲	بافر پر است
۳	تولیدکننده: (empty)wait برای عنصر C	−۱	۲	تولیدکننده مسدود می‌شود (در صف)
۴	مصرف‌کننده: (full)wait … برداشتن A	−۱	۱	یک اسلات آزاد شد
۵	مصرف‌کننده: (empty)signal	۰	۱	مقدار ≥ ۰ ← تولیدکننده بیدار می‌شود
۶	تولیدکننده ادامه می‌دهد، C را ذخیره و (full)signal می‌زند	۰	۲	بافر دوباره پر شد

نتیجه: مجموع empty + full همیشه برابر ظرفیت (۲) است مگر هنگامی که فرایندی مسدود شده باشد. تولیدکننده بدون هدررفت CPU در گام ۳ می‌خوابد و در گام ۵ بیدار می‌شود.

سؤال ۷. مقدار اولیه‌ی سمافور mutex برای انحصار متقابل چیست؟

الف) ۰ ب) ۱ ج) N د) ۲

پاسخ: گزینه‌ی «ب»؛ مقدار ۱ یعنی ناحیه‌ی بحرانی آزاد است و فقط یک فرایند از wait عبور می‌کند.

سؤال ۸. در پیاده‌سازی مسدودکننده، اگر مقدار یک سمافور ۳− باشد چه معنایی دارد؟

الف) سه منبع آزاد ب) سه فرایند مسدود در صف ج) خطا د) سه signal اضافی

پاسخ: گزینه‌ی «ب».

سؤال ۹. چرا جابه‌جاکردن wait(mutex) و wait(empty) در تولیدکننده بن‌بست می‌سازد؟

پاسخ: اگر بافر پر باشد، تولیدکننده ابتدا mutex را می‌گیرد و سپس روی wait(empty) می‌خوابد، در حالی که mutex را نگه داشته است. مصرف‌کننده برای آزادکردن اسلات باید wait(mutex) بزند و هرگز نمی‌تواند؛ پس هر دو برای همیشه منتظر می‌مانند.

مانیتور

مثال ۴: چرا while؟ (سبک Mesa)

بافر خالی است. مصرف‌کننده‌ی C۱ و C۲ و تولیدکننده‌ی P را در نظر بگیرید:

گام	رویداد	وضعیت
۱	C۱ تابع get() را صدا می‌زند: count = ۰ ← wait(notEmpty)؛ قفل آزاد می‌شود	C۱ در صف notEmpty می‌خوابد
۲	P تابع put(۱۰) را صدا می‌زند: count = ۱ و signal(notEmpty)	C۱ آماده می‌شود اما قفل هنوز دست P است
۳	P از مانیتور خارج می‌شود	قفل آزاد است
۴	C۲ تابع get() را صدا می‌زند و زودتر قفل را می‌گیرد: عنصر را برمی‌دارد، count = ۰	بافر دوباره خالی است
۵	C۱ قفل را دوباره می‌گیرد و شرط را آزمون می‌کند: count = ۰ ← درست	با while: C۱ دوباره منتظر می‌ماند (درست)؛ با if: C۱ از بافر خالی می‌خواند (خطا)

نتیجه: در سبک Mesa، بیدارشدن نخ تضمین نمی‌کند شرط هنوز برقرار باشد؛ بنابراین شرط باید در حلقه‌ی while دوباره آزموده شود.

سؤال ۱۰. چرا در مانیتور به سبک Mesa از while به‌جای if استفاده می‌شود؟

الف) سرعت بیشتر ب) ممکن است شرط بین signal و ادامه‌ی اجرا تغییر کند ج) الزام کامپایلر د) جلوگیری از stack overflow

پاسخ: گزینه‌ی «ب».

سؤال ۱۱. تفاوت signal سمافور و signal متغیر شرطی هنگامی که هیچ فرایندی منتظر نیست چیست؟

الف) تفاوتی ندارد ب) سمافور مقدار را زیاد می‌کند و به یاد می‌ماند؛ متغیر شرطی اثر را از دست می‌دهد ج) هر دو خطا می‌دهند د) سمافور اثر را از دست می‌دهد

پاسخ: گزینه‌ی «ب».

سؤال ۱۲. در کد Java این مقاله چرا از notifyAll() به‌جای notify() استفاده شده است؟

پاسخ: تولیدکننده‌ها و مصرف‌کننده‌ها روی یک صف انتظار مشترک می‌خوابند. notify() ممکن است نوع اشتباه را بیدار کند (مثلاً تولیدکننده‌ای را که بافر هنوز پر است)؛ آن نخ دوباره می‌خوابد و بیدارشدن به مصرف‌کننده‌ی واقعی نمی‌رسد و بن‌بست رخ می‌دهد. notifyAll() همه را بیدار می‌کند و هر نخ شرط خود را می‌آزماید.

خوانندگان و نویسندگان

مثال ۵: ردگیری گونه‌ی اول

رویدادها: R۱ در زمان ۰، R۲ در ۱، نویسنده‌ی W۱ در ۲، R۳ در ۳ وارد می‌شوند؛ R۱ در ۴، R۲ در ۵ و R۳ در ۶ خارج می‌شوند.

زمان	رویداد	read_count	rw_mutex	توضیح
۰	R۱ وارد می‌شود: اولین خواننده ← wait(rw_mutex)	۱	۰ (گرفته شد)	R۱ می‌خواند
۱	R۲ وارد می‌شود	۲	۰	R۱ و R۲ با هم می‌خوانند
۲	W۱: wait(rw_mutex)	۲	۰	W۱ مسدود می‌شود
۳	R۳ وارد می‌شود (اولویت با خواننده)	۳	۰	به خواننده‌ها می‌پیوندد
۴	R۱ خارج می‌شود	۲	۰	—
۵	R۲ خارج می‌شود	۱	۰	—
۶	R۳ خارج می‌شود: آخرین خواننده ← signal(rw_mutex)	۰	۱ ← W۱	W۱ بیدار شده و می‌نویسد

نتیجه: R۳ با وجود نویسنده‌ی منتظر از W۱ جلو افتاد؛ اگر خواننده‌های بیشتری می‌رسیدند، W۱ گرسنه می‌ماند. در نسخه‌ی منصفانه R۳ در `turnstile` می‌ایستاد.

سؤال ۱۳. در گونه‌ی اول مسئله کدام گروه ممکن است دچار گرسنگی شود؟

الف) خواننده‌ها ب) نویسنده‌ها ج) هیچ‌کدام د) هر دو

پاسخ: گزینه‌ی «ب».

سؤال ۱۴. اولین خواننده در گونه‌ی اول چه کاری انجام می‌دهد؟

الف) `signal(mutex)` را اجرا می‌کند ب) با `wait(rw_mutex)` نویسنده‌ها را بیرون می‌کند ج) منتظر نویسنده می‌ماند د) `read_count` را صفر می‌کند

پاسخ: گزینه‌ی «ب».

سؤال ۱۵. چگونه دروازه‌ی `turnstile` گرسنگی نویسنده را برطرف می‌کند؟

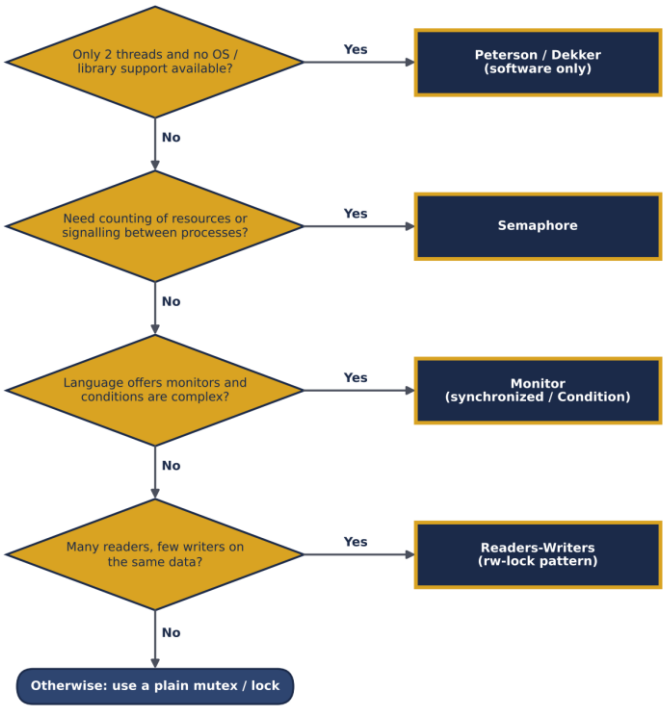
پاسخ: نویسنده پیش از `rw_mutex` سمافور `turnstile` را می‌گیرد و تا پایان نوشتن نگه می‌دارد. خواننده‌ی جدید باید از `turnstile` عبور کند و پشت نویسنده می‌ماند؛ بنابراین فقط خواننده‌های فعلی تمام می‌شوند و نویسنده بعد از آن‌ها وارد می‌شود.

۹. جمع‌بندی و راهنمای انتخاب

مطالعه‌ی پنج راهکار نشان می‌دهد هیچ‌کدام در همه‌ی معیارها برتر نیست. دکر و پترسون ثابت می‌کنند انحصار متقابل فقط با خواندن و نوشتن حافظه ممکن است، اما به‌دلیل انتظار مشغول و محدودیت دو فرایندی عملاً بیشتر برای آموزش و راستی‌آزمایی به کار می‌روند. سمافور ابزار همه‌منظوره‌ی سیستم‌عامل است که هم انحصار و هم هماهنگی میان فرایندها را فراهم می‌کند، اما خطاپذیر است. مانیتور با پنهان‌کردن قفل در یک ساختار، ایمن‌ترین و خواناترین گزینه در زبان‌هایی است که از آن پشتیبانی می‌کنند. مسئله‌ی خوانندگان و نویسندگان نیز نشان می‌دهد وقتی الگوی دسترسی خواندنی است، می‌توان با پذیرش هم‌روندی خواندن کارایی را بسیار بالا برد، به شرط آنکه گرسنگی را مدیریت کنیم.

جدول ۶ – پیشنهاد انتخاب بر اساس سناریو

سناریو	پیشنهاد	دلیل
آموزش انحصار متقابل بدون پشتیبانی OS	پترسون (یا دکر)	کوتاه‌ترین الگوریتم نرم‌افزاری درست
همگام‌سازی چند فرایند در هسته‌ی OS یا C/POSIX	سمافور	مسدودسازی، N فرایند، در همه‌ی سیستم‌عامل‌ها
برنامه‌نویسی در Python / C# / Java با شرط‌های پیچیده	مانیتور	قفل را زبان مدیریت می‌کند
داده‌ی خواندنی با نوشتن کم (کش، پایگاه‌داده)	خوانندگان–نویسندگان (منصفانه)	خواندن موازی، بدون گرسنگی



شکل ۱۵ – درخت تصمیم برای انتخاب سازوکار همگام‌سازی

پیام نهایی

در عمل برنامه‌نویس به‌جای نوشتن دکر یا پترسون از قفل‌های آماده‌ی کتابخانه (`pthread_mutex`، `synchronized`، `ReentrantReadWriteLock`) استفاده می‌کند؛ اما شناخت این پنج الگوریتم دلیل درستی آن قفل‌ها و خطاهای رایج (بن‌بست، گرسنگی، وارونگی اولویت) را روشن می‌سازد.

منابع

[۱] E. W. Dijkstra, “Solution of a Problem in Concurrent Programming Control,” Communications of the ACM, ۸(۹), p. ۵۶۹, ۱۹۶۵.

[۲] E. W. Dijkstra, “Cooperating Sequential Processes,” Technical Report EWD-۱۲۳, Technische Hogeschool Eindhoven, ۱۹۶۵.

[۳] G. L. Peterson, “Myths About the Mutual Exclusion Problem,” Information Processing Letters, ۱۲(۳), pp. ۱۱۵–۱۱۶, ۱۹۸۱.

[۴] C. A. R. Hoare, “Monitors: An Operating System Structuring Concept,” Communications of the ACM, ۱۷(۱۰), pp. ۵۴۹–۵۵۷, ۱۹۷۴.

[۵] P. J. Courtois, F. Heymans, D. L. Parnas, “Concurrent Control with ‘Readers’ and ‘Writers’,” Communications of the ACM, ۱۴(۱۰), pp. ۶۶۷–۶۶۸, ۱۹۷۱.

[۶] B. W. Lampson, D. D. Redell, “Experience with Processes and Monitors in Mesa,” Communications of the ACM, ۲۳(۲), pp. ۱۰۵–۱۱۷, ۱۹۸۰.

[۷] A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, ۱۰th ed., Wiley, ۲۰۱۸.

[۸] A. S. Tanenbaum, H. Bos, Modern Operating Systems, ۴th ed., Pearson, ۲۰۱۴.

[۹] M. Herlihy, N. Shavit, The Art of Multiprocessor Programming, ۲nd ed., Morgan Kaufmann, ۲۰۲۰.

[۱۰] A. B. Downey, The Little Book of Semaphores, Green Tea Press, ۲۰۱۶.